Unique readability: every lambda term has exactly one formation history

The Open Logic Project contributors

openlogic:lambda-calculus/syntax:local-snapshot-20261013; CC BY 4.0

Editorial note. All delivered mathematical text is the author's own, copied from the pinned Open Logic Project unit file sources/olp-lambda-calculus-syntax/pinned-original-source.tex; no statement or proof step was rewritten, reordered or removed.

Editorial note. The counted proof is the author's complete argument: all three formation cases are expanded in the span with their shape-lemma citations; no case is left to the reader.

Editorial note. The label prop:unq names the counted statement, and the proof environment immediately following it in the source is delivered verbatim as its proof.

Editorial note. The counted statement is carried by the author's own prop environment in the unit source.

Allowed standard prerequisites (editorial).

- Formation rules of terms. Syntax unit, definition \ollabel{defn:term} with rules \ollabel{defn:term-var} / term-abs / term-app. The three cases of the formation induction, keyed by which rule could have produced M.
- Terms begin with a variable or a parenthesis. Same unit, \ollabel{lem:term-start} (source line 1541). Case 3: excludes an abstraction reading of a term beginning with a parenthesis followed by a non-abstraction head.
- Applications begin with two parentheses or a parenthesis and a variable. Same unit, \ollabel{lem:app-start} (source line 1553). Case 2: an abstraction cannot also be built as an application.
- No proper initial part of a term is a term. Same unit, \ollabel{lem:initial} (statement at source line 1569; the unit leaves its proof as an exercise by induction on length). Case 3: two application readings would force one first argument to be a proper initial segment of the other.

Source reference. Original lemma lem-app-start: sources/olp-lambda-calculus-syntax/pinned-original-source.tex, source lines 1553–1553. Prerequisite pointer invoked at proof line(s) 1591; it names a standard fact of the same unit that the counted argument uses, and every step of the counted proof built on it is retained verbatim in the delivered body.

Source reference. Original lemma lem-initial: sources/olp-lambda-calculus-syntax/pinned-original-source.tex, source lines 1565–1565. Prerequisite pointer invoked at proof line(s) 1603; it names a standard fact of the same unit that the counted argument uses, and every step of the counted proof built on it is retained verbatim in the delivered body.

Source reference. Original lemma lem-term-start: sources/olp-lambda-calculus-syntax/pinned-original-source.tex, source lines 1541–1541. Prerequisite pointer invoked at proof line(s) 1598; it names a standard fact of the same unit that the counted argument uses, and every step of the counted proof built on it is retained verbatim in the delivered body.

Source reference. Original definition defn-term: sources/olp-lambda-calculus-syntax/pinned-original-source.tex, source lines 1489–1489. Prerequisite pointer invoked at proof line(s) 1586; it names a standard fact of the same unit that the counted argument uses, and every step of the counted proof built on it is retained verbatim in the delivered body.

Source reference. Original definition defn-term-var: sources/olp-lambda-calculus-syntax/pinned-original-source.tex, source lines 1492–1492. Prerequisite pointer invoked at proof line(s) 1586; it names a standard fact of the same unit that the counted argument uses, and every step of the counted proof built on it is retained verbatim in the delivered body.

Proposition 0.1 (Unique Readability)**.** *There is a unique formation for each term. In other words, if a term M is formed by a formation, then it is the only formation that can form this term.*

Proof. We prove this by induction on the formation of terms.

1. M is of the form x, where x is some variable. Since the results of abstractions and applications always start with parentheses, they cannot have been used to construct M; Thus, the formation of M must be a single step of defn-termdefn-term-var.

2. M is of the form $(\lambda[x][N])$, where x is some variable and N is a term. It could not have been constructed according to defn-termdefn-term-var, because it is not a single variable. It is not the result of an application, by lem-app-start. Thus M can only be the result of an abstraction on N. By inductive hypothesis we know that formation of N is itself unique.

3. M is of the form (PQ), where P and Q are terms. Since it starts with a parentheses, it cannot also be constructed by defn-termdefn-term-var. By lem-term-start, P cannot begin with λ, so (PQ) cannot be the result of an abstraction. Now suppose there were another way of constructing M by application, e.g., it is also of the form $(P'Q')$. Then P is a proper initial segment of P' (or vice versa), and this is impossible by lem-initial. So P and Q are uniquely determined, and by inductive hypothesis we know that formations of P and Q is unique.

□